Internship report

Alexandre Vannereux

1 Introduction

The internship was with Martin Pépin and Matthieu Dien as internship supervisors. The goal was to code and integrate pointing to the python library Usainboltz [3]. This library implements the Boltzmann method which allows fast uniform sampling on combinatorial class at a fixed size [4].

2 Background

Before explaining what I did during the internship, the basic of combinatorial theory and usainboltz must be established.

2.1 Combinatorial theory

Definition 1 (Combinatorial class) *A class of combinatorial structures is a pair $\mathcal{C} = \langle \mathscr{C}, |.|_{\mathcal{C}} \rangle$, where $\mathscr{C}$ is a finite or denumerable set and $|.|_{\mathcal{C}}$ is an application from $\mathscr{C}$ to $\mathbb{N}$.*

For $c \in \mathscr{C}$, $|c|_{\mathcal{C}}$ is the size of c. There must always be a finite number of elements of each size in $\mathscr{C}$.

The counting sequence $(C_n)_{n\in\mathbb{N}}$ associated with $\mathscr{C}$ is the number of elements of each size in $\mathscr{C}$. I.e. $\forall n \in \mathbb{N}, C_n = |\{c \in \mathscr{C} || c|_{\mathcal{C}} = n\}|$.

All combinatorial class are labelled or unlabelled, we will see further in the paper what that means.

Later on, we will indistinctively use $\mathscr{C}$ or $\langle \mathscr{C}, |.| \rangle$. We will sometimes simplify the notation for the size: $|.|$. When different class of combinatorial structures will be involved context can be used to differentiate their different size functions.

Definition 2 (Isomorphism) *Two combinatorial class are said to be isomorphic if they have the same number of elements of each size.*

We now introduce the two most frequently used and simple class of combinatorial structures. They are often used as base class to build more complex ones.

Definition 3 (Epsilon and atom) *The epsilon class $\epsilon = \langle \mathscr{E}, |.|_\epsilon \rangle$ is the class composed of only one element of size zero: $\mathscr{E} = \{e\}$ and $|e|_\epsilon = 0$.*

Very similar, the atom class is comprised of exactly one element of size one: $\mathcal{Z} = \langle \{z\}, |.|_{\mathcal{Z}} \rangle$ with $|z|_{\mathcal{Z}} = 1$.

We also introduce the zero class:

Definition 4 (Zero) *The zero class is the class containing no elements.*

We now define a number of operations on those objects:

Definition 5 (Product) *Given two class of combinatorial structures, $\mathcal{A} = \langle \mathscr{A}, |.|_{\mathscr{A}} \rangle$, $\mathcal{B} = \langle \mathscr{B}, |.|_{\mathscr{B}} \rangle$, their product noted $\mathcal{A} \times \mathcal{B}$ is the class $\langle \mathscr{A} \times \mathscr{B}, |.|_{\mathcal{A}\times\mathcal{B}} \rangle$ such that $\forall (a,b) \in \mathscr{A} \times \mathscr{B}, |(a,b)|_{\mathcal{A}\times\mathcal{B}} = |a|_{\mathcal{A}} + |b|_{\mathcal{B}}$.*

As mentionned earlier, we can simplify the notations for the size function: $|.|$ and referring as a class by its set: $\mathscr{C}$.

Definition 6 (Union) *Given two class of combinatorial structures, $\mathscr{A}$, $\mathscr{B}$, we define their union $\mathscr{A} + \mathscr{B}$ as follows. Let μ and μ' such that $\mu \neq \mu'$ and $|\mu| = |\mu'| = 0$, then $\mathscr{A} + \mathscr{B} = (\{\mu\} \times \mathscr{A}) \bigcup (\{\mu'\} \times \mathscr{B})$. This is equivalent to changing elements of $\mathscr{B}$ such that $\mathscr{A}$ and $\mathscr{B}$ are disjoint.*

This definition can be extended to the union of a finite or denumerable number of class of combinatorial structures as long as only a finite number of elements of size n appear in the union. For instance:

Definition 7 (Sequence) *The sequence construction noted $\mathscr{C} = Seq(\mathscr{A})$ is defined as $\mathscr{C} = \mathscr{E} + \mathscr{A} + \mathscr{A} \times \mathscr{A} + \mathscr{A} \times \mathscr{A} \times \mathscr{A} + ...$ To ensure that $\mathscr{C}$ is properly defined, $\mathscr{A}$ must not contain any element of size zero. Otherwise, there would be an infinite number of elements of same size.*

The two following constructions are only allowed in the labelled case.

Definition 8 (Labelled Set) *The set construction noted $\mathscr{C} = Set(\mathscr{A})$ is the class of sets elements of $\mathscr{A}$. It can be seen as the quotient set of $Seq(\mathscr{A})$ by the relation $A = (A_1, ..., A_n) \sim B = (B_1, ..., B_n)$ if and only if B is a permuation of A.*

Definition 9 (Labelled Cycle) *The cycle construction noted $\mathscr{C} = Cyc(\mathscr{A})$ is the class of cycles of elements of $\mathscr{A}$. It can be seen as the quotient set of $Seq(\mathscr{A})$ by the relation $A = (A_1, ..., A_n) \sim B = (B_1, ..., B_n)$ if and only if B is a cyclic permuta*

Those three constructions have equivalent definitions with bounds on their number of elements. For instance $Seq(\mathscr{A}, geq = 5)$ means that we consider sequences with at least 5 elements of the class $\mathscr{A}$.

Using only products, unions, sequences, sets, cycles and finite sets to build class offer a very limited range of possibilities close to the expression level of regular languages. We can use recursive definition to obtain a range closer to context-free grammar.

Definition 10 (specification) *A specification of a class of combinatorial structures $\mathscr{C}$ is a set of (possibly recursive) equations using only operations described above, atoms, epsilons and zeros such that $\mathscr{C}$ is the unique solution to the primary equation (up to an isomorphism).*

$\mathscr{C}$ is said to be specifiable if it admits a specification of finite size.

This definition means that for any specifiable class $\mathscr{C}$, an element $c \in \mathscr{C}$ is composed of n atoms. Due to aboves definitions, each of these atoms is distinct from the other.

An alternate definition for sequences is by giving it a specification: $\mathscr{C} = Seq(\mathscr{A})$ is equivalent to $\mathscr{C}$ is solution of $\mathscr{C} = \mathscr{E} + \mathscr{A} \times \mathscr{C}$. Then the specification of $\mathscr{C}$ would be this equation joined with the specification of $\mathscr{A}$.

We can now explain what is a labelled and an unlabelled class.

Definition 11 (Labelled and Unlabelled class) *A labelled class $\mathscr{C}$ is a class where for each element C of $\mathscr{C}$, the atoms of C have been distinguished by attributing a distrinct number from 1 to $|C|$. This is a proper definition due to the fact that each atom is distinct.*

In the case where such a mapping is not provided, we consider $\mathscr{C}$ to be an unlabelled class.

For more basic construction, labelled and unlabelled structures are very similar. When building more complex class, unlabelled class are harder to deal with. That's why unlabelled cycles (UCycle) and sets (PSet, MSet) have been ommited for now.

Now that we have combinatorial class and the capacity to build them, we can define an important tool to study them.

Definition 12 (ordinary and exponential generating functions) *Let $\mathscr{C}$ be an unlabelled combinatorial class, and (C_n) its counting sequence. The Ordinary Generating Function (OGF) of $\mathscr{C}$ is $C(z) = \sum_{n\in\mathbb{N}} z^n C_n$.*

Let $\mathscr{C}$ be a labelled combinatorial class, and (C_n) its counting sequence. The Exponential Generating Function (EGF) of $\mathscr{C}$ is $C(z) = \sum_{n\in\mathbb{N}} \frac{z^n C_n}{n!}$.

In both cases, we denote by ρ_C its radius of convergence.

A notation often used to obtain C_n from C is $[z^n]C(z) = C_n$.

The letter used for a combinatorial class will always be the same one used for its corresponding generating function. *OGF* will always be used for unlabelled structures and *EGF* for labelled structures.

Finally, we will often alternate between seeing C as a formal series or a power series. Context can be used to differentiate the two.

Property 1 (product, union and sequence on *OGF* and *EGF*) *Let $\mathscr{A}$ and $\mathscr{B}$ be two combinatorial class (labelled or unlabelled).*

- *If $\mathscr{C} = \mathscr{A} \times \mathscr{B}$ then $C(z) = A(z)B(z)$.*
- *If $\mathscr{C} = \mathscr{A} + \mathscr{B}$ then $C(z) = A(z) + B(z)$.*
- *If A has no objects of size 0 and $\mathscr{C} = Seq(\mathscr{A})$, then $C(z) = \frac{1}{1-A(z)}$.*
- *If A has no objects of size 0 and $\mathscr{C} = Set(\mathscr{A})$ (always in the label case), then $C(z) = \exp(A(z))$.*
- *If A has no objects of size 0 and $\mathscr{C} = Cyc(\mathscr{A})$ (always in the label case), then $C(z) = \ln(\frac{1}{1-A(z)})$.*

For the epsilon and atom class, EGF and OGF functions are identical:

- *$E(z) = 1$ (the epsilon class)*
- *$Z(z) = z$ (the atom class)*

It is important to note that there is no difference dealing with labelled and unlabelled generatings functions for now.

We can now talk about the Boltzmann model properly.

Definition 13 (Boltzmann model) *Let $\mathscr{C}$ a combinatorial class, C its generating function and $x \in]0, \rho_C[$.*

The boltzmann model of parameter x over $\mathscr{C}$ assign to each $\gamma \in \mathscr{C}$ the probability:

$$\mathbb{P}_{x,\mathscr{C}}(\gamma) = \frac{x^\gamma}{C(x)} \quad \textit{(unlabelled case)}$$

$$\mathbb{P}_{x,\mathscr{C}}(\gamma) = \frac{x^\gamma}{C(x)|\gamma|!} \quad \textit{(labelled case)}$$

$\mathbb{P}_{x,\mathscr{C}}$ can sometimes be written $\mathbb{P}_x$.

Despite the probability distribution of labelled and unlabelled generating functions being different, the associated samplers are almost identical.

Definition 14 (Boltzmann Samplers) *A boltzmann sampler ΓC for a class $\mathscr{C}$ is an algorithm that produces objects from $\mathscr{C}$ with respect to $\mathbb{P}_x$.*

Then, we note $\Gamma C(x) \in \mathscr{C}$, an output of this algorithm.

This paper will now focus on the boltzmann sampler introduced in [4].

Let $\mathscr{C}$ a specifiable combinatorial class, we now define the Boltzmann Sampler $\Gamma C(x)$ by induction on the specification of $\mathscr{C}$.

- If $\mathscr{C}$ is finite, then this is just taking a random element from C according to a finite distribution which we assume can easily be done.
- If $\mathscr{C} = \mathscr{A} + \mathscr{B}$. We define $p = \frac{A(x)}{A(x)+B(x)} = \frac{A(x)}{C(x)}$. The probability that an element $\gamma \in \mathscr{C}$ is an element of A is $\mathbb{P}(\gamma \in \mathscr{A}) = p$. Therefore, $\Gamma C(x)$ is $\Gamma A(x)$ with probability p and $\Gamma B(x)$ with probability $1 - p$.
- If $\mathscr{C} = \mathscr{A} \times \mathscr{B}$. Then we observe that $\mathbb{P}_{x,\mathscr{C}} = (\mathbb{P}_{x,\mathscr{A}}, \mathbb{P}_{x,\mathscr{B}})$. Therefore $\Gamma C(x)$ is $(\Gamma A(x), \Gamma B(x))$.
- If $\mathscr{C} = Seq(\mathscr{A})$. We can again rewrite this equality as: $\mathscr{C} = \{\epsilon\} + \mathscr{A} \times \mathscr{C}$ and use the sampler for product and unions.
- If $\mathscr{C} = Set(\mathscr{A})$ then the number of elements of $\mathscr{A}$ in the set follow a Poisson law of parameter $A(x)$. Due to the labelling in $\mathscr{C}$, we are guaranted to not have two equals elements in a set, therefore our algorithm for $\Gamma C(x)$ is: take k a random value sampled from the poisson law of parameter $A(x)$, make k independant calls to $\Gamma A(x)$, return the resulting set.
- If $\mathscr{C} = Cycle(\mathscr{A})$ like for the set, we need to identify a law for the number of elements in the cycle. Let K a random variable which follow the law $\mathbb{P}(K = k) = -\frac{A(x)^k}{log(1-A(x))k}$. Then, we have a very similar algorithm to the set: take k a random value sampled according to X, make k independant calls to $\Gamma A(x)$, return the resulting cycle.

In each case, when we consider labelled case, the algorithm also attributes labels properly which we assume can be done easily.

The proof for the correctness of those samplers is done in [4].

Property 2 (Sampler complexity) *Given an oracle computing values of all OGF present in the specification of $\mathscr{C}$ in $O(1)$ the sampler is linear in the size of its output in terms of number of basic real-arithmetic operations $(+, -, \times, \div)$.*

The proof has be partially done in [4] and in [1]

From this we deduce a rejection sampler.

Definition 15 *(Rejection Sampler) For $\mathscr{C}$ a combinatorial class, $n \in \mathbb{N}$,and $\epsilon > 0$ we define the rejection sampler $\mu C(n, \epsilon)$:*

Choose $x \in]0, \rho_C[$, sample $\gamma \in \mathscr{C}$ according to $\mathbb{P}_x$, repeat until $|\gamma| \in [n(1-\epsilon), n(1+\epsilon)]$. Return γ.

This sampler relies heavily on the distribution probability on sizes, therefore we need to study the associated random variable $N = |\gamma|$.

Property 3 (probability of N and some notations.) *The probability distribution on N is as follows: $\forall n \in \mathbb{N}, \mathbb{P}_x(N = n) = \frac{C_n x^n}{C(x)}$.*

From this, we can easily deduce the expected value, the second moment and standard deviation:

- $v_1(x) = \mathbb{E}_x(N) = \frac{xC'(x)}{C(x)}$.
- $v_2(x) = \mathbb{E}_x(N^2) = \frac{x^2C''(x)+xC'(x)}{C(x)}$.

- $\sigma(x) = \sqrt{v_2(x) - v_1(x)^2}$.

Proof 1 *Immediate due to the definition of* $\mathbb{P}_x$.

The formula for the first and second moment are of particular importance. The first determines a certain target size and the second control the Bolztmann Sampler range.

Is is also important to note that $\mathbb{E}_x[N]$ is a strictly increasing function of x as long as $\mathscr{C}$ contains at least two elements of different sizes. Indeed,

$$\mathbb{E}_x[N]' = \frac{\sigma^2(x)}{x}$$

and the standard deviation ($\sigma(x)$ here) is always strictly positive when the associated random variable can take different values.

Therefore, it is possible in many situation to find for any $n \in \mathbb{N}$ a unique value x such that $\mathbb{E}_x[N] = n$.

2.2 The library Usainboltz (Uniform SAmplINg with BOLTZmann)

The library Usainboltz created in 2019 implements the Boltzmann method in a broad set of cases. This library is coded in python, cython and c++ and has three main modules:

- The module grammar implements tools to create and manipulate specifications.
- The module oracle evaluates generating functions and tune a value x (i.e. find a solution to $\mathbb{E}_x[N] = n$).
- The module generator allows when provided with a grammar and an optional oracle (if not provided, the generator will create its own oracle) to sample elements from the combinatorial class associated with the grammar.

When sampling, the generator doesn't build the structure from the root to the leaves, instead it calls builders functions provided by the user each time an auxiliary class is reached.

Example 1 (The height of a binary tree) *To better understand how the usainboltz library works, we provide the simple example of a generator sampling the height of a binary tree.*

```
from usainboltz import *

e, z = Epsilon(), Atom()
T = RuleName("T")#rulenames correspond to auxiliary class in a grammar.

G = Grammar({T: e + z*T*T}) #Creates the grammar for binary trees.
gen = Generator(T) #Creates the associated generator, the oracle could be provided
#by writing: Generator(T, oracle = oracle)

def height_leaf(_):
    return 0

def height_node(tp: tuple):
    #the builder has been called on the left and right child
    #of the node, the first argument corresponds to the atom z.
    (_, left, right) = tp
    return max(left, right) + 1

gen.set_builder(T, union_builder(height_leaf, height_node))
#The builder needed for T receive as argument a special tuple which correspond to the union.
#To hide this aspect, usainboltz provides a function to obtain builders for unions,
#when the element on the left is chosen (e in this context), height_leaf is called
```

```
#otherwise height_node is called.

res = gen.sample((10, 20))
#Finally, we sample a binary tree containing between 10 and 20 nodes, the result, is an
#integer corresponding to the height of the sample tree.
```

3 Vector pointing

My task for this internship was to implement the pointing method in the Usainboltz library.

3.1 Pointing in combinatorial theory

As explained above, when a class has a specification, we have the guarantee that each element of size n has exactly n atoms, from this we can deduce a new operator on class:

Definition 16 (Pointing) *Let $\mathscr{C}$ be a specifiable combinatorial class, we consider the class $\mathscr{C}^\bullet = \{(C, c)|C \in \mathscr{C}, c$ is an atom of $C\}$ such that $|(C, c)|_{\mathscr{C}^\bullet} = |C|_{\mathscr{C}}$.*

This operator corresponds to the derivation (and multiplication by z) of the generating function, both in the labelled and unlabelled case: $C^\bullet(z) = zC'(z)$.

As explained in [4] and [1], pointing can offer advantages when manipulating Boltzmann Samplers. Those samplers rely heavily on tuning which is not always possible when the expected value is bounded, pointing can sometimes remove the boundary on the expected value.

3.2 Grammar pointing

Since the Boltzmann method use specifications, we need to be able to point a class given its specification. There are two main ways to do so, we can first introduce a new operator *PointOp* which can be used like any other operator. Even though this solution is appealing this is not what has been done in practice, indeed contrary to other operator $Union, Sequence, etc...$, *PointOp* doesn't have a clean recursive algorithm.

Instead we use an alternate definition of pointing. We can define pointing as an operation on specifications:

Definition 17 (Pointing) *We consider the specification of C_1:*

$$\begin{aligned}&\{\\ &C_1 &&= \Psi_1(C_1, ..., C_n)\\ &...C_n &&= \Psi_n(C_1, ..., C_n)\\ &\}\end{aligned}$$

where $\Psi_1, ..., \Psi_n$ are functions which only use the operator and basic class: $Atom(), Zero(), Epsilon(), +, \times, Seq, Set, Cycle$.

Then, a specification for $C_1^\bullet$ is:

$$\begin{aligned}&\{\\ &C_1^\bullet &&= \Psi_1^\bullet(C_1^\bullet, C_1, ..., C_n^\bullet, C_n)\\ &C_1 &&= \Psi_1(C_1, ..., C_n)\\ &...\\ &C_n^\bullet &&= \Psi_n^\bullet(C_1^\bullet, C_1, ..., C_n^\bullet, C_n)\\ &C_n &&= \Psi_n(C_1, ..., C_n)\\ &\}\end{aligned}$$

For a function Ψ, $\Psi^\bullet$ is defined recursively by

- $Atom()^\bullet = Atom()$
- $Epsilon()^\bullet = Zero()^\bullet = Zero()$
- $(A_1 + ... + A_n)^\bullet = A_1^\bullet + ... + A_n^\bullet$
- $(A_1 \times ... \times A_n)^\bullet = A_1^\bullet \times A_2 \times ... \times A_n + ... + A_1 \times ... \times A_{n-1} \times A_n^\bullet$
- $(Seq(A))^\bullet = Seq(A) \times A^\bullet \times Seq(A)$
- $(Set(A))^\bullet = A^\bullet \times Set(A)$
- $(Cycle(A))^\bullet = A^\bullet \times Seq(A)$

What has been said above for pointing remain true when using this definition of pointing.
The following algorithm points a grammar in usainboltz.

```
def GrammarPoint(g: Grammar):
    #g.rules is a mapping of rulenames to their associated rule in the specification.
    new_rules = g.rules.copy()
    for (rulename, rule) in g.rules.items():
        new_rules[Point(rulename)] = Point(rulename)
    g.rules = new_rules

#Some shortcuts are allowed: None−1 = None; (i−j) = max(0, i−j). Very useful for sizes constraints.
def Point(S):
    switch S:
        case Atom():
            return Atom()
        case Epsilon() | Zero():
            return Zero()
        case Union(R1, ..., Rn):
            return Union(Point(R1), ..., Point(Rn))
        case Product(R1, ..., Rn):
            return Union(
                Product(Point(R1), R2, ..., Rn),
                ...,
                Product(R1, ..., Point(Rn))
        case RuleName(name):
            return RuleName(name + "_P")
        case Set(R, leq = i, geq = j):
            return Product(R, Set(Point(R), leq = i−1, geq = j−1))
        case Cycle(R, leq = i, geq = j):
            return Product(R, Seq(Point(R), leq = i−1, geq = j−1))
        case Seq(R, leq = None, geq = None):
            return Union(Product(Seq(R, eq = 0), Point(R), Seq(R)))
        case Seq(R, leq = i, geq = j):
            #with i != None.
            return Union(
                Product(Seq(R, eq = 0), Point(R), Seq(R, leq = i−1, geq = j−1)),
                Product(Seq(R, eq = 1), Point(R), Seq(R, leq = i−2, geq = j−2)),
                ...,
                Product(Seq(R, eq = i−1), Point(R), Seq(R, leq = 0, geq = j−i)))
        case Seq(R, leq = None, geq = j):
            #with j != None
            return Union(
                Product(Seq(R, eq = 0), Point(R), Seq(R, leq = None, geq = j−1)),
                ...,
```

```
        Product(Seq(R, eq = j−2), Point(R), Seq(R, leq = None, geq = 1)),
        Product(Seq(R, geq = j−1), Point(R), Seq(R)))
```

3.3 Builder Pointing

One important element in usainboltz, as mentionned above, is the usage of builders. The user can provide building functions which will be called each time an auxiliary class is reached. However, when a class is pointed the associated builder can't be used on the pointed class as the underlying structure has changed. The first solution is to require the user to provide a new builder function, this is not optimal as the user needs to know what the pointed grammar looks like. Moreover, pointing a grammar can be made automatic and the user might not be aware that the grammar has been pointed.

The second solution (the one I implemented in Usainboltz) is to derive pointed builders (i.e. builders for pointed class) from the builders provided by the user and the grammar structure. To do so, we need to first define what is an acceptable tuple for a grammar, then we can provide and prove the correctness of our algorithm. This has been done in details in [2], we offer here a quick overview of the main idea behind this theory.

To differentiate python tuple from mathematical couple, we use the prefix Tp when noting a python tuple.

Definition 18 (Acceptable tuple for a grammar) *A context Γ is a couple $\Gamma = (G, M)$ where G is a grammar and M is a mapping such that for all rulenames A present in G, $\mathcal{B} = M(A)$ is a function which takes as argument, an acceptable tuple for A.*

For Γ a context, R a rule appearing in a production rule of G and t a tuple, we say that t is acceptable for R, and we write $\Gamma : R \vdash t$, if and only if this proposition can be obtained from a set of inference rules. Here we only show a couple of them, see [2] for an exhaustive description of the inference rules.

$$\frac{}{\Gamma : Atom() \vdash Tp(Atom(),)} \; Atom$$

$$\frac{\Gamma : R_1 \vdash t_1 \quad ... \quad \Gamma : R_n \vdash t_n}{\Gamma : R_1 \times ... \times R_n \vdash Tp(t_1, ..., t_n)} \; Product$$

$$\frac{A \text{ is an auxiliary class associated with the rule } R \text{ in } G \quad \Gamma : R \vdash t}{\Gamma : A \vdash Tp(M(A)(t),)} \; Simplification$$

We also define the *UnpointTp* algorithm which will be used to derive pointed builders:

```
def UnpointTp(t: tuple, S: Rule) −> tuple:
    switch S:
        case Union(R1, ..., Rn):
            Tp(i, tu) = t #This can raise an error if t has the wrong format.
            return Tp(i, UnpointTp(Ri, tu, stop_at_rulename))
        case Product(R1, ..., Rn):
            Tp(i, Tp(t1, ..., tn)) = t
            return Tp(t1, ..., UnpointTp(Ri, ti, stop_at_rulename), ..., tn)
        case RuleName(A):
            return t
        case Cycle(R) | Set(R):
            Tp(t1, Tp(t2, ..., tn))
            return Tp(Unpoint(R, t1, stop_at_rulename), t2, ..., tn)
        case Seq(R):
            Tp(_, Tp(Tp(t_left_1, ..., t_left_n), t_pointed, Tp(t_right_1, ..., t_right_m))) = t
            return Tp(t_left_1, ..., t_left_n, UnpointTp(R, t_mid, stop_at_rulename), t_right_1, ..., t_right_m)
        case Epsilon() | Zero():
            raise Exception("Error")
```

This is important to note that this algorithm stops when an auxiliary class is reached.
Finally, we define a pointed context:

Definition 19 (Pointed context) *For a context $\Gamma = (G, M)$, the pointed context $\Gamma^\bullet$ is $\Gamma^\bullet = (G^\bullet, M^\bullet)$. For $A \to R$ a production rule of G, $M^\bullet(A) = M(A)$ and $M^\bullet(A^\bullet)(t^\bullet) = M(A)(UnpointTp(t^\bullet, R))$ where $t^{b\bullet}$ is an acceptable tuple for $R^\bullet$.*

For this definition to be correct, we need to make sure that $UnpointTp(t^\bullet, R)$ is an acceptable tuple for A.

Theorem 1 (Correctness of $UnpointTp$) *Let $\Gamma = (G, M)$ be a context, R a rule appearing in G and $t^\bullet$ a tuple. $\Gamma^\bullet : R^\bullet \vdash t^\bullet$ if and only if $UnpointTp(t^\bullet, R)$ is correctly defined and $\Gamma : R \vdash UnpointTp(t^{v\bullet}, R)$.*

The proof has be done in [2].

4 Cycle Pointing

As mentionned previously, some unlabelled constructions have been omitted. We will now see three new unlabelled constructions:

Definition 20 (MSet) *If $\mathscr{C}$ is an unlabelled class, then $MSet(\mathscr{C})$ is the class of sets of elements of $\mathscr{C}$ where repetitions are allowed.*

Definition 21 (UCycle) *If $\mathscr{C}$ is an unlabelled class, then $UCycle(\mathscr{C})$ is defined identically as for the labelled case (in fact, we could use the same notation Cyc).*

Definition 22 (PSet) *If $\mathscr{C}$ is an unlabelled class, then $Pset(\mathscr{C})$ is the class of sets of elements of $\mathscr{C}$ where no repetitions are allowed.*

This last construction is not implemented in Usainboltz, indeed we need to be able to eliminate elements which already appear in the set during the sampling which is made impossible by the builders.

References

[1] Vannereux A. Pointing method in boltzmann sampler (dans le dossier du fichier tex, c'est mon mémo, vive les lapins!). 2026.

[2] Vannereux A. tuple conversion in usainboltz for pointing. 2026.

[3] Pepin M. and Dien M. Uniform sampling with boltzmann. *25th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing*, 2023.

[4] Flajolet P., Duchon P., Louchard G., and Schaeffer G. Boltzmann samplers for the random generation of combinatorial structures. *Combinatorics, Probability and Computing*, pages 577–625, 2004.